

Day 3

REST API Knowledge

🎯 **Goal:** Understand REST APIs deeply so you can connect your AI agents to ANY external service — weather, databases, LLMs, payment systems, search engines, and more.

 Part 1: Learn (30 minutes)

3.1 What is a REST API?

REST = **RE**presentational **State** **T**ransfer

An API (Application Programming Interface) is a **menu** for software. Just like a restaurant menu tells you what you can order, an API tells you what data you can request.

REAL WORLD:
You → Menu → Waiter → Kitchen → Food comes back

SOFTWARE:
Your Code → API Docs → HTTP Request → Server → JSON Response comes back

AI AGENTS:
Agent → Tool Definition → API Call → External Service → Result comes back

Every AI agent is basically a program that reads API menus and places orders automatically.

3.2 REST API Structure — The URL Anatomy

Every API call has a structured URL:

https://api.example.com/v1/users/123/posts?sort=date&limit=10

Protocol

Base URL

Version

Path
(Resource)

Query Params
(Filters)

Breaking it down:

Part	Purpose	Example
Protocol	HTTP or HTTPS (secure)	https://
Base URL	The server address	api.openai.com
Version	API version (for compatibility)	/v1/
Path/Resource	What data you want	/chat/completions
Query Params	Filters and options	?model=gpt-4o&temp=0.7

Real AI API Examples:

OpenAI: https://api.openai.com/v1/chat/completions

Gemini: https://generativelanguage.googleapis.com/v1/models/gemini-pro:generateContent

GitHub: https://api.github.com/repos/Sushant-Dadheech/agent-ai-learning

Weather: https://api.openweathermap.org/data/2.5/weather?q=Jaipur

3.3 HTTP Methods — The 5 Actions

REST APIs use HTTP methods to define WHAT you want to do:

Method	Action	Agent Use Case	Example
GET	Read/Fetch data	Agent checks weather, reads emails	GET /users/123
POST	Create/Send data	Agent sends prompt to LLM, creates ticket	POST /chat/completions
PUT	Update (replace entirely)	Agent updates a full user profile	PUT /users/123
PATCH	Update (partial)	Agent changes just the email	PATCH /users/123
DELETE	Remove data	Agent deletes a scheduled task	DELETE /tasks/456

The CRUD Pattern:

```
CREATE  → POST
READ   → GET
UPDATE → PUT / PATCH
DELETE → DELETE
```

In Day 1, you used GET (fetch joke) and POST (create post). Today you'll learn all 5.

3.4 HTTP Status Codes — The Server's Response Language

You learned basic codes in Day 1. Here's the FULL picture:

The 5 Families:

Range	Family	Meaning	Agent Behavior
1xx	Informational	Request received, processing	Rare — just wait
2xx	Success	Request worked!	Use the response data
3xx	Redirect	Resource moved	Follow the redirect
4xx	Client Error	YOUR request is wrong	Fix and retry (or don't)
5xx	Server Error	SERVER has a problem	Wait and retry

The Codes You'll See Most:

```
SUCCESS:
  200 OK           → Data returned successfully (GET)
  201 Created      → New resource created (POST)
  204 No Content   → Action done, no data to return (DELETE)

CLIENT ERRORS (your fault):
  400 Bad Request  → Your JSON is malformed or missing fields
  401 Unauthorized → API key is missing or wrong
  403 Forbidden    → Valid key, but no permission for this action
  404 Not Found    → The endpoint/resource doesn't exist
  422 Unprocessable → Data format is right but values are invalid
  429 Too Many Reqs → Rate limited! Slow down. (VERY common with LLMs)

SERVER ERRORS (their fault):
  500 Internal Error → Server crashed
  502 Bad Gateway    → Server behind a proxy crashed
  503 Unavailable    → Server is overloaded (common during GPT launches)
```

3.5 Headers — Metadata for Every Request

Headers are key-value pairs that carry extra info with every request:

```
headers = {
  "Content-Type": "application/json",    # I'm sending JSON
  "Accept": "application/json",         # I want JSON back
  "Authorization": "Bearer sk-abc123...", # My API key
  "User-Agent": "MyAgent/1.0",          # Who I am
  "X-Request-ID": "uuid-here"           # Track this request
}
```

Headers You'll Use in Every Agent:

Header	Purpose	Example
Content-Type	Format of data you're sending	application/json
Accept	Format you want back	application/json
Authorization	Your API key/token	Bearer sk-...
User-Agent	Identifies your app	MyAIAgent/1.0

3.6 Authentication — Proving Who You Are

APIs need to know WHO is calling them. Three main patterns:

Pattern 1: API Key in Query Parameter

```
GET https://api.weather.com/forecast?apikey=abc123&city=Jaipur
```

- Simplest method
- Less secure (key visible in URL/logs)
- Used by: Weather APIs, some free APIs

Pattern 2: API Key in Header (Most Common for AI)

```
headers = {  
  "Authorization": "Bearer sk-proj-abc123..."  
}  
requests.post("https://api.openai.com/v1/chat/completions", headers=headers)
```

- More secure (key not in URL)
- Used by: **OpenAI, Gemini, Claude, GitHub, most professional APIs**

Pattern 3: OAuth 2.0 (Complex but Powerful)

```
1. User clicks "Login with Google"  
2. Redirected to Google's login page  
3. User approves access  
4. Your app receives an access_token  
5. Use that token for future API calls
```

- Most secure
- Used by: Google, LinkedIn, Twitter, Slack
- Needed when accessing USER data (not just your own)

*For AI agents, you'll mostly use **Pattern 2** (Bearer tokens).*

3.7 Request & Response Bodies — The Data

Request Body (what you SEND):

```
{  
  "model": "gpt-4o",  
  "messages": [  
    {"role": "system", "content": "You are a helpful assistant."},  
    {"role": "user", "content": "What is an AI agent?"}  
  ],  
  "temperature": 0.7,  
  "max_tokens": 500  
}
```

Response Body (what you GET BACK):

```
{
  "id": "chatcmpl-abc123",
  "object": "chat.completion",
  "model": "gpt-4o",
  "choices": [
    {
      "message": {
        "role": "assistant",
        "content": "An AI agent is a program that can..."
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 25,
    "completion_tokens": 150,
    "total_tokens": 175
  }
}
```

Notice the pattern: You send structured JSON → You get structured JSON back. Every LLM API follows this same pattern.

3.8 Query Parameters — Filtering & Pagination

When APIs return lots of data, you use query params to control what you get:

```
GET /api/posts?page=2&limit=20&sort=date&order=desc
      |           |           |           |
      Page number Items   Sort by  Direction
                per page  field
```

Pagination Patterns:

Pattern 1: Page-based (most common)

```
GET /posts?page=1&limit=20    → items 1-20
GET /posts?page=2&limit=20    → items 21-40
GET /posts?page=3&limit=20    → items 41-60
```

Pattern 2: Cursor-based (used by modern APIs)

```
GET /posts?limit=20                → first 20 items + next_cursor
GET /posts?limit=20&cursor=abc123  → next 20 items + next_cursor
GET /posts?limit=20&cursor=def456  → next 20 items + next_cursor=null (done!)
```

AI agents often need to paginate through search results, emails, or database records.

3.9 Rate Limiting — Don't Get Banned

APIs limit how many requests you can make:

```
OpenAI GPT-4o:    500 requests/minute
GitHub API:       5000 requests/hour
Free Weather API: 60 requests/minute
Google Gemini:     15 requests/minute (free tier)
```

How to Handle Rate Limits:

```
import time

def call_with_backoff(func, max_retries=5):
    for attempt in range(max_retries):
        response = func()
        if response.status_code == 429:
            # Read the Retry-After header if available
            wait = int(response.headers.get("Retry-After", 2 ** attempt))
            print(f"Rate limited. Waiting {wait}s...")
            time.sleep(wait)
        else:
            return response
    raise Exception("Max retries exceeded")
```

3.10 Key Takeaways — Day 3 Cheat Sheet

URL STRUCTURE:

`https://api.example.com/v1/resource?param=value`

HTTP METHODS:

GET	= Read	POST	= Create
PUT	= Replace	PATCH	= Update
DELETE	= Remove		

STATUS CODES:

2xx = Success 4xx = Your fault 5xx = Their fault
 429 = Rate limited (most important for agents!)

AUTH:

API Key in header: Authorization: Bearer YOUR_KEY

AGENT RULE:

Every external tool an agent uses = one REST API call
 More APIs you know = more powerful your agent becomes

🕒 Part 2: Daily Lab Task (30 minutes)

Lab: "Build a Multi-API Client"

Objective: Build a Python script that talks to 3 different APIs using different methods and authentication patterns.

Task 1: GET with Query Parameters — Weather API

```
# Fetch weather for a city using query parameter authentication
# API: https://wttr.in/{city}?format=json (free, no key needed!)
# Extract: temperature, humidity, weather description
```

Task 2: GET with Headers — GitHub API

```
# Fetch your own repo info using the GitHub API
# URL: https://api.github.com/repos/Sushant-Dadheech/agent-ai-learning
# Headers: Accept: application/vnd.github+json
# Extract: stars, forks, description, created date
```

Task 3: POST with JSON Body — JSONPlaceholder

```
# Create a new resource by sending JSON in the body
# URL: https://jsonplaceholder.typicode.com/posts
# Send: title, body, userId
# Verify: 201 status code and returned ID
```

Task 4: Pagination — Fetch Multiple Pages

```
# Fetch 3 pages of users from JSONPlaceholder
# URL: https://jsonplaceholder.typicode.com/posts?_page=1&_limit=5
# Loop through pages 1, 2, 3
# Print how many total items you collected
```

Task 5: Error Handling + Status Code Analysis

```
# Hit different endpoints and handle each status code:
# 1. Valid request (200)
# 2. Non-existent resource (404)
# 3. Post with wrong content-type (400)
# Print a clear message for each status code
```

Expected Output:

```
--- DAY 3 LAB: Multi-API Client ---

TASK 1: Weather API
  City: Jaipur
  Temperature: 35C
  Humidity: 60%
  Condition: Partly cloudy

TASK 2: GitHub API
  Repo: agentic-ai-learning
  Description: Learning Agentic AI from scratch...
  Stars: 0 | Forks: 0
  Created: 2026-07-28

TASK 3: POST Request
  Status: 201 Created
  New post ID: 101

TASK 4: Pagination
  Page 1: 5 items
  Page 2: 5 items
  Page 3: 5 items
  Total collected: 15 items

TASK 5: Error Handling
  200 → Success: Data received
  404 → Not Found: Resource doesn't exist
  [Test appropriate error scenarios]

LAB COMPLETE!
```

Completion Checklist

- ☐ Task 1: Fetched weather data with query params
- ☐ Task 2: Used GitHub API with proper headers
- ☐ Task 3: Sent POST request with JSON body
- ☐ Task 4: Implemented pagination across 3 pages
- ☐ Task 5: Handled multiple status codes correctly
- ☐ Updated my_progress.json to Day 3

Coming Tomorrow — Day 4

LLM Basics — What are Large Language Models? How do transformers work? Tokens, embeddings, attention mechanism, and temperature/top-p parameters. You'll finally start working with AI models directly!

"Every tool an AI agent uses is just a REST API call in disguise." 🛠️ Complete the lab and push your `day3_lab.py` to GitHub!